

Control flow

A hands-on course in building agents by building their harness

Lesson 3

Control flow

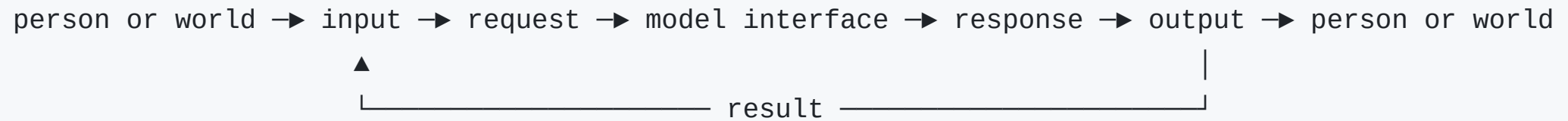
Control flow is how information flows between the other four primitives.

They all sit inside it

They all sit inside it: it decides what runs, in what order, and whether to go again.

The path one pass takes

Here's the path one pass takes, with the pieces from Lessons 1 and 2:



Input and the model interface

Input gathers what goes into the request.

The model interface sends it and gets the response back.

Output, and a tool's result

Output handles the response: it shows it to a person, or runs a tool.

A tool's result is input again.

The end of the path

Control flow decides what happens at the end of the path: the result goes back around, a person gets a turn, or everything stops.

How you arrange them

How you arrange them decides what you've built:

Run once

Run once. Input, call, output, stop. That's Lesson 2.

A chat loop

A chat loop. Call, show the response, hand back to the person, repeat. A chatbot.

A workflow

A workflow. Your code decides the steps: call the model to draft, then to check, then to fix.

An agent loop

An agent loop. Call, and while the model asks for tools, run them, send the results back, and call again. Hand back when it stops asking.

There's no correct one

There's no correct one. They differ in who decides what happens next: in a workflow, your code does; in an agent, the model does.

A workflow

A workflow is predictable and easy to test, but only handles what you planned for.

An agent

An agent handles what you didn't plan for, and costs you predictability.

It **sequences**

Whatever the shape, control flow does two things. It **sequences** the other primitives, passing what one produced to the next.

It terminates

And it **terminates**: it decides when to stop, or when to hand back to a person.

A loop with no clear way to end is a bill with no clear way to end.

The worked example: `quark.py`

Here's Lesson 2's input, model interface and output inside quark's loop. It's the whole of `quark.py`:

```
import subprocess, sys
from anthropic import Anthropic

# — model interface —————
client = Anthropic()
MODEL = "claude-sonnet-5-5"
def call(**request): return client.messages.create(model=MODEL, **request)

# — output: the one tool —————
tools = [{"name": "bash", "description": "Run shell command – the whole system is in reach", "input_schema": {"type": "object", "properties": {"cmd": {"type": "string"}}, "required": ["cmd"]}]}
```

quark.py : input

```
# — input —————
def read(prompt):                                # input: from a person
    while True:
        print(prompt, end="", flush=True)
        line = sys.stdin.readline()
        if not line: return "/q"                # end of input (Ctrl-D): nothing more is coming
        if line.strip(): return line.rstrip("\n") # Enter on an empty line: a fresh prompt, as in a terminal
        prompt = "> "
input = " ".join(sys.argv[1:]) or read("> ")
if input == "/q": sys.exit()
chat = len(sys.argv) < 2
```

quark.py: control flow

```
# — control flow —————  
messages = [{"role": "user", "content": input}]  
  
while True:  
    output = call(max_tokens=16384, tools=tools, messages=messages).content  
    messages.append({"role": "assistant", "content": output})  
  
    input = []  
    for block in output:                                # output: show text, run tool requests  
        if block.type == "text":  
            print(block.text)  
        if block.type == "tool_use":  
            print(f"$ {block.input['cmd']}")  
            done = subprocess.run(block.input["cmd"], shell=True, stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True)  
            print(done.stdout)  
            input.append({"type": "tool_result", "tool_use_id": block.id, "content": done.stdout or f"(exit {done.returncode})"}) # input: from the world
```

quark.py: handing back

```
if input:
    messages.append({"role": "user", "content": input})
    continue
if not chat or (input := read("\n> ")) == "/q":
    break
messages.append({"role": "user", "content": input})
```

Lesson 2's code, plus `chat`

Everything above `# — control flow —` is Lesson 2's, with one addition to input: `chat`, which is true when nothing came in on the command line.

```
chat = len(sys.argv) < 2
```

The new section is the loop.

```
# — control flow —————
```

while True:

```
while True:  
    output = call(max_tokens=16384, tools=tools, messages=messages).content
```

while True: means go again. Each pass is one call to the model.

messages

```
messages = [{"role": "user", "content": input}]
```

messages is how a result reaches the next call. The first entry is the input that started the run.

What gets appended

```
messages.append({"role": "assistant", "content": output})
```

```
messages.append({"role": "user", "content": input})
```

Right after each call, the model's `output` is appended; if the model asked for tools, the `input` they produced is appended after it.

The simplest form of context

Each request carries everything so far.

That growing list is the simplest form of context, working memory. Deciding what really goes in it is Lesson 4.

`if input: ... continue`

```
if input:  
    messages.append({"role": "user", "content": input})  
    continue
```

`if input: ... continue` is the agent loop. If the model asked for anything, what came back goes in and the loop calls again.

No input from the world

No input from the world is termination. The model decides it's done by not asking for a tool. quark doesn't count steps or look for a magic word.

chat

chat gives one loop two modes. With input on the command line, quark runs until the model stops asking for tools, then exits.

```
chat = len(sys.argv) < 2
```

chat : the chat mode

With none, it's a chat: when the model's turn ends, it hands back to you with `>`, `read()` gathers your next input, and `/q` (or Ctrl-D) quits.

```
if not chat or (input := read("\n> ")) == "/q":  
    break  
messages.append({"role": "user", "content": input})
```

No step limit

Notice what the loop doesn't have: a step limit. quark trusts the model to finish.

That's a choice, and if the model never stops asking, quark never stops.

Run `quark.py`

From the root of the repo:

```
uv run lessons/03-control-flow/quark.py "which quark.py in the lessons folder is the longest? answer in one sentence"
```

One run

Here's one run:

```
$ find . -ipath '*lessons*' -name 'quark.py' -exec wc -l {} + 2>/dev/null | sort -n
  9 ./lessons/01-model-interface/quark.py
 33 ./lessons/02-input-and-output/quark.py
 46 ./lessons/03-control-flow/quark.py
234 ./lessons/04-context/quark.py
322 total
```

The longest is `./lessons/04-context/quark.py`, at 234 lines.

One run: the first call

That's two calls. The first asked for a command.

```
$ find . -ipath '*lessons*' -name 'quark.py' -exec wc -l {} + 2>/dev/null | sort -n
```

One run: the second call

Its result went back, and the second call answered from what it found.

```
9 ./lessons/01-model-interface/quark.py
33 ./lessons/02-input-and-output/quark.py
46 ./lessons/03-control-flow/quark.py
234 ./lessons/04-context/quark.py
322 total
```

The longest is `./lessons/04-context/quark.py`, at 234 lines.

Chat with it

Run it with no input to chat with it instead.

What else control flow can be

What else control flow can be: quark runs one agent loop with no limit, but this primitive holds more in other harnesses.

Its shape, and when it stops

- **Its shape.** A single call, a chat loop, a workflow, an agent loop, agents that hand tasks to other agents, a loop that wakes on a message or a schedule.
- **When it stops.** When the model stops asking, after a number of steps, at a spending limit, when the model declines, when a person says stop.

Who it hands back to, and what runs at once

- **Who it hands back to.** A person, another program, or no one: it runs until the task is done.
- **What runs at the same time.** One model call at a time, or several at once.

LangGraph

It can be a product on its own. [LangGraph](#) is built around this primitive: you lay out the steps and the paths between them as a graph, and it runs them.

control_flow.py

Here's control flow that does more of that, in `control_flow.py`:

```
import subprocess, sys
from anthropic import Anthropic

client = Anthropic()
tools = [{"name": "bash", "description": "Run shell command", "input_schema": {"type": "object", "properties": {"cmd": {"type": "string"}}, "required": ["cmd"]}]}
MAX_STEPS = 10

task = " ".join(sys.argv[1:]) or input("> ")
messages = [{"role": "user", "content": task}]
```

control_flow.py: the loop and the refusal

```
for step in range(1, MAX_STEPS + 1):
    reply = client.messages.create(model="claude-sonnet-5-5", max_tokens=16384, tools=tools, messages=messages)
    messages.append({"role": "assistant", "content": reply.content})
    if reply.stop_reason == "refusal":
        print("[stopped: the model declined]")
        break
```

control_flow.py: the tools run

```
results = []
for block in reply.content:
    if block.type == "text":
        print(block.text)
    if block.type == "tool_use":
        print(f"$ {block.input['cmd']}")
        done = subprocess.run(block.input["cmd"], shell=True, stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True)
        print(done.stdout)
        results.append({"type": "tool_result", "tool_use_id": block.id, "content": done.stdout or f"(exit {done.returncode})"})
```

control_flow.py: done, or out of steps

```
if not results:
    print(f"[done in {step} steps]")
    break
messages.append({"role": "user", "content": results})
else:
    print(f"[stopped: hit the {MAX_STEPS}-step limit]")
```

When it stops

When it stops. The loop runs at most `MAX_STEPS` calls. It also stops when the model is done, or when it declines with a `refusal`, and it says which.

Who it hands back to

Who it hands back to. No one. There's no chat mode; it runs the task and exits.

Run `control_flow.py`

Run it:

```
uv run lessons/03-control-flow/control_flow.py "which folder in lessons has the most files? answer in one English sentence"
```

One run

Here's one run:

```
$ cd lessons 2>/dev/null && for d in */; do echo "$(find "$d" -type f | wc -l) $d"; done | sort -rn | head -5; pwd
5 02-input-and-output/
4 03-control-flow/
3 04-context/
3 01-model-interface/
/home/user/building-agents/lessons
```

The `02-input-and-output` folder in `lessons` has the most files, with 5.
[done in 2 steps]

Two steps

Two steps: one command, then the answer, and the loop says why it stopped.

MAX_STEPS set to 1

With **MAX_STEPS** set to 1, the same harness stops after the first command, before the model sees its result:

```
$ git --version  
git version 2.43.0  
  
[stopped: hit the 1-step limit]
```

Building effective agents

Anthropic's [Building effective agents](#) names the workflows that come up most:

Prompt chaining

- **Prompt chaining.** A fixed sequence of calls, each working on the last one's output, with checks in code between them if you want.

Routing

- **Routing.** One call sorts the input, and your code sends it down the path built for that kind.

Parallelization

- **Parallelization.** Calls run at the same time, either splitting a task into independent parts or asking the same thing several times and combining the answers.

Orchestrator–workers

- **Orchestrator–workers.** One call breaks the task into pieces as it goes, other calls do the pieces, and the results are combined.

Evaluator–optimizer

- **Evaluator–optimizer.** One call produces something, another judges it against the task, and the loop repeats until the judge passes it.

Two things to keep straight

Two things to keep straight when you read it. The article's building block, the *augmented LLM*, is a model with retrieval, tools and memory, and that isn't control flow: retrieval and memory are context, and tools are output.

Routing stays control flow

And routing stays control flow even when the path it picks is a cheaper model: that's a decision about the work, unlike Lesson 1's backup model, which only steps in when the first is down and belongs to the model interface.

workflow.py

`quark.py` and `control_flow.py` are both agents: the model decides what happens next. Here's a workflow instead, an evaluator–optimizer, in `workflow.py`:

```
import sys
from anthropic import Anthropic

client = Anthropic()

def ask(prompt):
    reply = client.messages.create(model="claude-sonnet-5-5", max_tokens=2048, messages=[{"role": "user", "content": prompt}])
    return next(b.text for b in reply.content if b.type == "text")
```

`workflow.py`: the draft

```
task = " ".join(sys.argv[1:]) or input("> ")  
draft = ask(f"Do this task. Reply with only the result.\n\n{task}")
```

workflow.py: judge and rewrite

```
for round in range(1, 4):
    print(f"--- draft {round} ---\n{draft}\n")
    verdict = ask(f"Task:\n{task}\n\nDraft:\n{draft}\n\nCheck the draft against every requirement in the task. If it meets all of them, reply with only PASS. Otherwise list what to fix.")
    if verdict.strip() == "PASS":
        print("[evaluator: pass]")
        break
    print(f"--- evaluator ---\n{verdict}\n")
    draft = ask(f"Task:\n{task}\n\nDraft:\n{draft}\n\nFeedback:\n{verdict}\n\nRewrite the draft to fix everything in the feedback. Reply with only the new draft.")
else:
    print("[stopped: 3 rounds without a pass]")
```

Who decides

Who decides. Your code. There are no tools and no loop the model steers: the order is always draft, judge, rewrite, judge.

When it stops

When it stops. When the judging call replies `PASS` , or after three rounds.

The prompts

The prompts. Each call gets one prompt built from the task and the last draft. That's the simplest possible context; the point here is the order of the calls.

Run `workflow.py`

Run it:

```
uv run lessons/03-control-flow/workflow.py "write one sentence about agent harnesses in which every word has exactly five letters, at least eight words long"
```

One run: the first draft

Here's one run:

```
--- draft 1 ---  
Agent rigs wrap model calls while tools guide small tasks.
```

One run: the evaluator

```
--- evaluator ---
```

```
**Fails:** Two words are not five letters long:
```

- "rigs" has 4 letters.
- "wrap" has 4 letters.

The other words (Agent, model, calls, while, tools, guide, small, tasks) are all five letters, and the sentence is long enough (10 words).

```
**Fix:** Replace "rigs" and "wrap" with five-letter words. For example:
```

```
"Agent frame holds model parts while tools guide small tasks."
```

Every word in this version has exactly five letters (Agent, frame, holds, model, parts, while, tools, guide, small, tasks), and it has 10 words.

One run: the second draft

```
--- draft 2 ---  
Agent frame holds model parts while tools guide small tasks.  
  
[evaluator: pass]
```

The first draft missed

The first draft missed, the judging call said exactly why, and the rewrite passed. Every step was one your code laid out in advance.

The rule

The rule: control flow sequences the other four primitives and decides when to stop. Arranged differently, the same four pieces are a single call, a chatbot, an agent, or a workflow.

What control flow never does

Notice what control flow never does. Getting the request to the model and the response back is the model interface. Gathering what goes in is input, and handling the response is output.

Context is not control flow

What each request holds, including that growing `messages` list and the prompts in `workflow.py`, is context. Control flow only decides what runs, in what order, and when to stop.

What's missing

What's missing: this is an agent, but it knows nothing. Not where it is, not what day it is, not who it is, not what you told it yesterday.

messages grows

And **messages** grows every pass; in a long session it will outgrow what the model can read. Something has to decide what the model sees. That's context.

Next: Context

Lesson 4